📘 INFORME COMPLETO: CRUD EN ASP.NET CORE MVC

Modelo-Vista-Controlador | C# | Razor Views | Almacenamiento en memoria

📑 Índice de Contenido

- 1. Introducción al Patrón MVC
- 2. Estructura del Proyecto
- 3. Modelo: Clase Datos (código completo)
- 4. Vistas completas: Index, Agregar, Editar, Eliminar, Layout
- 5. Controlador HomeController (código íntegro)
- 6. Representación de Pantallas (imágenes simuladas)
- 7. Flujo CRUD paso a paso
- 8. Análisis técnico y funcionamiento
- 9. Buenas prácticas y mejoras sugeridas
- 10. Conclusión

1. Introducción al Patrón MVC

🎯 ARQUITECTURA **MVC (Model-View-Controller)** separa la aplicación en tres capas:

- **Modelo**: Datos y lógica de negocio (clase `Datos`).
- **Vista**: Interfaz de usuario (archivos .cshtml con HTML/Razor).
- **Controlador**: Gestiona peticiones, interactúa con el modelo y selecciona la vista.

✅ Beneficios: mantenibilidad, reutilización, escalabilidad y separación de responsabilidades.

2. Estructura del Proyecto

```
EjercicioCompleto/
├── Models/
│   └── Datos.cs                    # Modelo
├── Views/
│   ├── Home/
│   │   ├── Index.cshtml            # Login
│   │   ├── Agregar.cshtml          # Listado + formulario alta
│   │   ├── Editar.cshtml           # Edición
│   │   └── Eliminar.cshtml         # Confirmación borrado
│   └── Shared/
│       └── _Layout.cshtml          # Plantilla común
└── Controllers/
    └── HomeController.cs           # Controlador principal
```

3. Modelo: Clase Datos (código completo)

```
// Datos.cs
namespace EjercicioCompleto.Models
{
    public class Datos
    {
        public string? usuario_ { get; set; }      // Nombre de usuario (identificador)
        public string? contraseña_ { get; set; }   // Contraseña en texto plano
        public string? habilitado_ { get; set; }   // "on" = habilitado, null = deshabilitado
    }
}
```

Nota: El checkbox envía `"on"` al servidor cuando está marcado, de lo contrario `null`.

4. Vistas completas (código fuente íntegro)

📄 _Layout.cshtml (Plantilla principal)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - EjercicioCompleto</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
</head>
<body>
    <header>
        <nav>
            <a asp-area="" asp-controller="Home" asp-action="Index">Inicio sesión</a>
            <a asp-controller="Home" asp-action="Agregar">Agregar</a>
        </nav>
    </header>
    <div class="container">
        <main>@RenderBody()</main>
    </div>
    <footer>&copy; 2026 - EjercicioCompleto</footer>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

🔐 Index.cshtml (Inicio de sesión)

```
@{
    ViewData["Title"] = "Inicio sesión";
}
<form asp-controller="Home" asp-action="validar" method="post">
    <div>
        <label>Usuario</label>
        <input type="text" name="txtUsuario" />
    </div>
    <div>
        <label>Contraseña</label>
        <input type="password" name="txtContraseña" />
    </div>
    <div>
        <input type="submit" value="Ingresar" />
    </div>
</form>
<h1 style="color:red;">@ViewBag.respuesta</h1>
```

📋 Agregar.cshtml (Listado + creación)

```
@{
    ViewData["Title"] = "Agregar";
}
<h1>Agregar</h1>
<form asp-controller="Home" asp-action="Nuevo" method="post">
    <input type="text" name="txtUsuarioNuevo" placeholder="Usuario" />
    <input type="password" name="txtContraseñaNuevo" placeholder="Contraseña" />
    <label>Habilitado</label>
    <input type="checkbox" name="chkHabilitado" />
    <input type="submit" value="Añadir" />
</form>

<table border="1">
    <tr><th>USUARIO</th><th>CONTRASEÑA</th><th>ESTADO</th><th>EDITAR</th><th>ELIMINAR</th></tr>
    @foreach (var usuarios in ViewBag.respuesta)
    {
        <tr>
            <td>@usuarios.usuario_</td>
            <td>@usuarios.contraseña_</td>
            <td>@(usuarios.habilitado_ == "on" ? "HABILITADO" : "DESHABILITADO")</td>
            <td>
                <form asp-action="Editar" method="post">
                    <input type="hidden" name="txtEditarUsuario" value="@usuarios.usuario_" />
                    <input type="hidden" name="txtEditarContraseña" value="@usuarios.contraseña_" />
                    <input type="hidden" name="chkEditarHabilitado" value="@usuarios.habilitado_" />
                    <input type="submit" value="Editar" />
                </form>
            </td>
            <td>
                <form asp-action="Eliminar" method="post">
                    <input type="hidden" name="txtUsuarioEliminar" value="@usuarios.usuario_" />
                    <input type="submit" value="Eliminar" />
                </form>
            </td>
        </tr>
    }
</table>
```

📝 Editar.cshtml

```
@{
    ViewData["Title"] = "Editar";
}
<h1>Editar</h1>
<form asp-controller="Home" asp-action="EditarDatos" method="post">
    <label>Usuario (no editable)</label>
    <input type="text" name="txtUsuarioNuevo" value="@ViewBag.UsuarioEditado" readonly />

    <label>Contraseña</label>
    <input type="password" name="txtContraseñaNuevo" value="@ViewBag.ContraseñaEditado" />

    <label>Habilitado</label>
    <input type="checkbox" name="chkHabilitadoNuevo" @ViewBag.HabilitadoEditado />

    <input type="submit" value="Editar registro" />
</form>
```

🗑️ Eliminar.cshtml

```
@{
    ViewData["Title"] = "Eliminar";
}
<h1>Eliminar</h1>
<h3 style="color:red;">¿Realmente desea eliminar al usuario @ViewBag.datoUsuario?</h3>
<div>
    <form asp-action="confirmaVolver">
        <input type="submit" value="Cancelar" />
    </form>
    <form asp-action="confirmaEliminar">
        <input type="hidden" name="txtUsuarioDatoEliminar" value="@ViewBag.datoUsuario" />
        <input type="submit" value="Confirmar" />
    </form>
</div>
```

5. Controlador HomeController (código íntegro)

```
using EjercicioCompleto.Models;
using Microsoft.AspNetCore.Mvc;

namespace EjercicioCompleto.Controllers
{
    public class HomeController : Controller
    {
        static List<Datos> listado = new List<Datos>();

        public IActionResult Index() => View();

        public IActionResult validar(string txtUsuario, string txtContraseña)
        {
            if (txtUsuario != null && txtContraseña != null)
            {
                var traeDatos = listado.Find(l => l.usuario_ == txtUsuario);
                if (traeDatos == null)
                    ViewBag.respuesta = "Usuario ingresado no existe en el sistema";
                else if (traeDatos.contraseña_ != txtContraseña)
                    ViewBag.respuesta = "Contraseña incorrecta";
                else if (traeDatos.habilitado_ != "on")
                    ViewBag.respuesta = "Usuario no se encuentra HABILITADO";
                else
                    ViewBag.respuesta = "Bienvenido usuario: " + txtUsuario;
            }
            else
                ViewBag.respuesta = "usuario vacio";

            return View("Index");
        }

        public IActionResult Agregar()
        {
            ViewBag.respuesta = listado;
            return View();
        }

        public IActionResult Nuevo(string txtUsuarioNuevo, string txtContraseñaNuevo, string chkHabilitado)
        {
            listado.Add(new Datos()
            {
                usuario_ = txtUsuarioNuevo,
                contraseña_ = txtContraseñaNuevo,
                habilitado_ = chkHabilitado
            });
            ViewBag.respuesta = listado;
            return View("Agregar");
        }

        public IActionResult Editar(string txtEditarUsuario, string txtEditarContraseña, string chkEditarHabilitado)
        {
            ViewBag.UsuarioEditado = txtEditarUsuario;
            ViewBag.ContraseñaEditado = txtEditarContraseña;
            ViewBag.HabilitadoEditado = (chkEditarHabilitado == "on") ? "checked" : "";
            return View();
        }

        public IActionResult EditarDatos(string txtUsuarioNuevo, string txtContraseñaNuevo, string chkHabilitadoNuevo)
        {
            var editaDatos = listado.Find(l => l.usuario_ == txtUsuarioNuevo);
            if (editaDatos != null)
            {
                editaDatos.contraseña_ = txtContraseñaNuevo;
                editaDatos.habilitado_ = chkHabilitadoNuevo;
            }
            ViewBag.respuesta = listado;
            return View("Agregar");
        }

        public IActionResult Eliminar(string txtUsuarioEliminar)
        {
            ViewBag.datoUsuario = txtUsuarioEliminar;
            return View();
        }
```

```
        public IActionResult confirmaVolver()
        {
            ViewBag.respuesta = listado;
            return View("Agregar");
        }

        public IActionResult confirmaEliminar(string txtUsuarioDatoEliminar)
        {
            var eliminaDatos = listado.Find(l => l.usuario_ == txtUsuarioDatoEliminar);
            if (eliminaDatos != null)
                listado.Remove(eliminaDatos);
            ViewBag.respuesta = listado;
            return View("Agregar");
        }
    }
}
```

6. Representación visual de las pantallas (simulando imágenes 1.png al 9.png)

Basado en las capturas originales del proyecto.

📸 PANTALLA 1: INDEX (LOGIN VACÍO)

EjercicioCompleto Inicio sesión Agregar

Usuario [campo vacío]

Contraseña [campo vacío]

Ingresar

📸 PANTALLA 2: AGREGAR SIN USUARIOS

EjercicioCompleto Inicio sesión Agregar

Añadir Usuario [ ]

Añadir Contraseña [ ]

Habilitado ☐

Añadir

USUARIO	CONTRASEÑA	ESTADO	EDITAR	ELIMINAR
No hay usuarios				

📸 PANTALLA 3: AGREGAR CON USUARIOS (A,B,C)

USUARIO	CONTRASEÑA	ESTADO	EDITAR	ELIMINAR
a	a	DESHABILITADO	Editar	Eliminar
b	b	HABILITADO	Editar	Eliminar
c	c	DESHABILITADO	Editar	Eliminar

📸 PANTALLA 4: EDITAR (USUARIO B)

Usuario `b (readonly)`

Contraseña `b`

Habilitado ☐

Editar registro

📸 PANTALLA 5: USUARIO B AHORA HABILITADO

USUARIO	CONTRASEÑA	ESTADO
a	a	DESHABILITADO
b	b	HABILITADO
c	c	DESHABILITADO

📸 PANTALLA 6: ELIMINAR (CONFIRMACIÓN USUARIO C)

¿Realmente desea eliminar al usuario c?

Cancelar Confirmar

📸 PANTALLA 7: LISTA TRAS ELIMINAR C

USUARIO	CONTRASEÑA	ESTADO
a	a	DESHABILITADO
b	b	HABILITADO

📸 PANTALLA 8: INTENTO LOGIN USUARIO A (DESHABILITADO)

Usuario `a`

Contraseña `a`

Ingresar

Usuario no se encuentra HABILITADO

📸 PANTALLA 9: LOGIN EXITOSO USUARIO B

Usuario b

Contraseña b

Ingresar

Bienvenido usuario: b

7. Flujo CRUD paso a paso

- **CREATE**: Formulario Agregar → POST /Home/Nuevo → listado.Add() → redirige a Agregar.
- **READ**: Vista Agregar recibe listado en ViewBag.respuesta y lo muestra en tabla.
- **UPDATE**: Botón Editar envía datos ocultos → Editar.cshtml → EditarDatos() modifica.
- **DELETE**: Botón Eliminar → Eliminar.cshtml confirmación → confirmaEliminar() remueve.

8. Análisis técnico y funcionamiento

🔍 CLAVE **ViewBag**: objeto dinámico para pasar datos entre controlador y vista. **Inputs hidden**: mantienen el identificador (usuario) en edición/eliminación. **Checkbox**: mapea a string "on"/null. **Lista estática**: los datos persisten solo en memoria mientras la app está activa.

9. Buenas prácticas y mejoras sugeridas

- ✅ **Seguridad:** Hash de contraseñas con BCrypt.
- ✅ **Persistencia:** Reemplazar lista estática por Entity Framework + SQL Server.
- ✅ **Identificador:** Usar Id autoincremental en lugar del nombre de usuario.
- ✅ **Validaciones:** DataAnnotations (`[Required]` , `[StringLength]`).
- ✅ **Manejo de errores:** Try-catch y mensajes amigables.

10. Conclusión

Este informe ha presentado el código completo y el diseño de pantallas de una aplicación CRUD funcional bajo el patrón MVC en ASP.NET Core. Se incluyen todas las vistas, el controlador con la lógica de negocio y el modelo de datos. Además, se han representado visualmente las 9 pantallas que muestran el flujo desde el login hasta la administración de usuarios. Es una base sólida para entender MVC y puede extenderse fácilmente con buenas prácticas de seguridad y persistencia real.